Pontificia Universidad
JAVERIANA
Cali

Proyect
Discrete Mathematics and Its Applications

November 4, 2025

Important: For the development of this project, you must form groups of **4 students** within your own class. Each member of the group will be required to demonstrate their individual contribution to the overall project by showing understanding of both the concepts involved and the implementation of the group's proposed solution. Failure to do so will result in the lowest possible grade for that student.

The N-Queens Problem

The **N-Queens problem** is a classical example in computer science and recreational mathematics. It consists of placing N queens on an $N \times N$ chessboard so that no two queens can attack each other. This means that no two queens can share the same row, column, or diagonal.

In chess, a queen can attack another one if they share the same row, column, or diagonal.

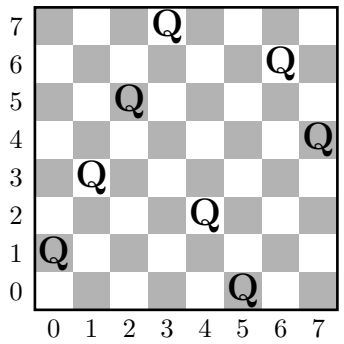

Figure 1: Example of a valid configuration for the 8-Queens problem.

Solving this problem is not only a logical and combinatorial challenge, but also serves as an introduction to advanced topics such as *backtracking algorithms*, state-space search, and optimization.

However, for the purposes of this course, we are not going to dive into the famous problem of finding a valid solution from scratch for any $N \times N$ board. Instead, the focus of this project will be to develop a program capable of performing the following tasks when given a specific configuration of N queens on an $N \times N$ board:

- Determine whether the configuration is valid or not (i.e. No queen can attack another queen).
- Determine the cardinality of the set of all possible configurations of N queens on a $N \times N$ board (both valid and invalid).

Validity Verification for an N-Queens Configuration

To verify whether a configuration of queens is valid, we can mathematically describe the positions of the queens and the conditions that must hold so that no two attack each other.

If we represent the position of the queen i as (x_i, y_i), then two queens i and j do not attack each other if: they are not in the same row ($x_i \neq x_j$), not in the same column ($y_i \neq y_j$) and not on the same diagonal (let us call this diagonal condition $D(i, j)$, where $D(i, j)$ denotes that queen i and queen j DO share a diagonal).

$$\neg D(i, j)$$

$$\forall i, j \cdot \; i, j \in \{0, 1, 2, \ldots, n-1\} \; \wedge \; x_i \neq x_j \; \wedge \; y_i \neq y_j \; \wedge \; \neg D(i, j) \tag{1}$$

Equation (1) expresses the necessary and sufficient condition for a N-Queens configuration on a $N \times N$ board to be valid.

$$\begin{aligned} x_i &\neq x_j && \text{(no shared row)} \\ y_i &\neq y_j && \text{(no shared column)} \\ D(i) &\neq D(j) && \text{(no shared diagonal)} \end{aligned}$$

In the project, one of the program's requirements is that it must be able to verify this condition for every pair of queens.

Sample Space Analysis for an N x N Board

In Probability and Statistics, the term **sample space** refers to the set of **all** possible outcomes of a random experiment. That is, it contains every potential result that could occur. In our case, the sample space corresponds to the set of all possible configurations of N queens on an $N \times N$ chessboard.

However, as is often the case, we are not interested in listing every single possible configuration, but rather in knowing *how many* there are. Determining the total number of distinct configurations is equivalent to finding the **cardinality** of the set of all possible configurations. This can be achieved by identifying certain key properties of these configurations.

Combinatorial or **counting techniques** are particularly useful in such cases because they allow us to compute the cardinality of large sets without having to enumerate each element individually.

In this project, part of your task is to determine which counting technique is most appropriate for this problem and to implement it correctly in a programming language.

Project Requirements

For this project, you will implement the algorithm for verifying a N-queens proposed solution. You will use the `Python` programming language to do so, while also following the next requirements:

- The program must be contained inside of a single Python file named *n-queens.py*
- The file must start with a comment right below the header where you will specify the following information:

```
"""
Discrete Mathematics and Its Applications
-------------------------------------------------
Members:
Class: (Either A or B)
"""
```

- For the implementation of the program you are only allowed to use the following libraries (if needed) apart from `Python`'s basic functionalities: `collections` , `heapq` , `queue`, `math` and `stdin`.

- Time and Space Complexity for your program will not be taken into account for grading, however, it is required that your algorithm has a polynomial runtime of, at most, n^4 per case ($T(n) \in O(n^4)$, with n being the number of queens for a given case).
- Since the program should be contained inside a single file, good programming practices will not be heavily taken into account for grading, however you must document the program leaving the right amount of commentaries throughout the code explaining each section of it.
- The section of your program that determines whether a given configuration of n-queens is valid or invalid should be encapsulated into a single function named *nqueens()*. This function may have as many input parameters as necessary but it must only a have single boolean value as output, representing if the configuration is valid (True) or not (False).
- Similarly, the section of your code that computes the total number of possible configurations must be encapsulated into a single function named *configurations()*. This function will only receive a single parameter n that represents the number of queens for the case and should output a single integer value c that represents the total amount of configurations for n-queens on a $n \times n$ board.
- Your program must have a *main()* function which will handle reading data from the terminal and process it as the input of several N-queens configurations, where you should perform both of the tasks listed in section **1**.

  The format for the terminal input is as follows: The first line of input contains a single number m, with $1 \leq m \leq 1000$, that denotes the number of N-queens cases to process. Afterwards, in the first line of each case there will be a single number n, with $n \leq 25$, that denotes the number of queens for the case. Next, there will be n lines, each of which will contain two numbers x, y (separated by a comma and a space), with $x \geq 0$ and $y \geq 0$, that represent the coordinates of each queen on the board (see ***Figure 1*** for further clarification). Finally, each case ends with a line where $x = -1$ and $y = -1$. The last case will also end with $-1, -1$.

Example Inputs

1. Example input from terminal for test case in ***Figure 1***:

```
1
8
0, 1
1, 3
2, 5
3, 7
5, 0
7, 4
6, 6
4, 2
-1, -1
```

   (Notice that the queens may be provided in any order)

2. Example input with more than one test case

```
2
8
0, 1
1, 3
2, 5
3, 7
5, 0
7, 4
6, 6
4, 2
-1, -1
9
0, 0
2, 1
4, 2
6, 3
8, 4
1, 5
3, 6
5, 8
6, 7
-1, -1
```

(Note that for n-queens, both the x and y positions belong to the interval $[0, n-1)$)

- As for the outputs of your program, you are required to present the results in **precisely** the next format. For the i-th case you should print in the terminal:

```
Case {i}:
Satisfies {n}-queen(s) problem? -> {answer}
Total Possible Configurations for {n}-queen(s): {total} configurations
```

There should be a blank line between each case. For numbers bigger than 10^{10} you must print them in scientific notation with a precision of the first 5 digits. For this purpose you shall include the next function into your program.

```
from math import log10, floor
#Place this import inside the file header
#(The file header is the start of the file where
#you put all the imports)

#Returns the scientific notation as a string.
def sn_format(n: int) -> str:
    signo = "-" if n < 0 else ""
    n_abs = abs(n)
    e = floor(log10(n_abs))
    mantisa_int = round(n_abs / 10**(e - 4))  # keeps first 5 digits
    mantisa_str = str(mantisa_int)

    if len(mantisa_str) > 5:
        mantisa_str = mantisa_str[:5]
        e += 1

    return f"{signo}{mantisa_str[0]}.{mantisa_str[1:]} x 10^{e}"
```

Example Outputs

1. Example output from terminal for test case in ***Figure 1***:

```
Case 1:
Satisfies 8-queen(s) problem? -> Yes
Total Possible Configurations for 8-queen(s): 4426165368 configurations
```

2. Example output with more than one test case

```
Case 1:
Satisfies 8-queen(s) problem? -> Yes
Total Possible Configurations for 8-queen(s): 4426165368 configurations

Case 2:
Satisfies 9-queen(s) problem? -> No
Total Possible Configurations for 9-queen(s): 2.6089 x 10^11 configurations
```

Research Questions

Finally, in addition to the python file, you must submit a *pdf* file where you must answer the following questions:

1. Find a function $g : \mathbb{N} \times \mathbb{N} \to F_1$, where F_1 is the set of all lineal functions with slope $= 1$, that maps each position in the board (x, y) to its ascending diagonal (The ascending diagonal is the one that goes up from left to right)
$$F_1 = \{f : \mathbb{R} \to \mathbb{R} \mid f(x) = 1x + b,\ b \in \mathbb{R}\}$$

2. Find a function $h : \mathbb{N} \times \mathbb{N} \to F_{-1}$, where F_{-1} is the set of all lineal functions with slope $= -1$, that maps each position in the board (x, y) to its descending diagonal (The descending diagonal is the one that goes down from left to right)
$$F_{-1} = \{f : \mathbb{R} \to \mathbb{R} \mid f(x) = -1x + b,\ b \in \mathbb{R}\}$$

3. Suppose we restrict the domain of both g and h to being only the set of positions of queens from a valid configuration of the problem. Show why g and h need to be one-to-one under these constraints.

4. Using your answers from previous questions, provide a logical equivalence for the logical proposition $D(i, j)$.

5. The P v.s. NP problem is one of the 7 millennium problems stated by the Clay Mathematics Institute in the year 2000, from which only one of them has been resolved. Watch the following videos about the P v.s. NP problem:

   - https://www.youtube.com/watch?v=UR2oDYZ-Sao
   - https://www.youtube.com/watch?v=1x4VbYerGsA

   In your own words, what is the importance of solving the P v.s NP problem?

6. Use the definition of the big O notation shown in the second video to develop an equivalent logical proposition for it.

7. Given your implementation of your program in `Python` and the new definition you constructed in the last question, argue why the run-time of your algorithm is $O(n^4)$.